Slipbox

A Curated, Agent-Operated Knowledge Base
for Unstructured Data

Technical Whitepaper — Rev. 1.4 — August 2026

Abstract. *Slipbox is a knowledge-base architecture that inverts the dominant retrieval paradigm: instead of ingesting raw documents cheaply and spending intelligence at query time, it invests intelligence at write time — distilling every captured document into atomic, human-reviewed notes placed in a linearly ordered store — so that retrieval becomes layered, auditable, and nearly trivial. The system is operated by an LLM agent, lives entirely in a local git repository of plain-text files, uses a single-file embedded vector index, and retains original documents in cold storage (`source/.attachments/`) that enables future re-extraction with better models. It is best understood as an alternative to the* ***LLM-maintained wiki*** *— the pattern of an agent owning a directory of generated markdown and rewriting it as sources arrive — differing from it on the single axis from which everything else follows: Slipbox's notes are immutable and human-gated, so knowledge accrues by* ***addition under review*** *rather than by* ***rewriting under automation****. This paper defines the scope of the system, explains its full processing lifecycle, analyses its retrieval properties, compares it with contemporary tooling and in detail with the LLM-maintained wiki, and specifies implementation details — including `synthesis/`, a materialised view of dated documents that cite atoms and are collected rather than rewritten.*

1 Scope

Slipbox targets a specific, deliberately narrow problem: building a **lifetime knowledge substrate** for an individual or a small trusted group, fed by a continuous stream of unstructured material — web articles, documents, images, media — arriving through conversational channels (e.g. a Telegram-connected agent gateway).

In scope:

- Capture of arbitrary unstructured sources into a durable inbox.
- Distillation of sources into atomic notes (one idea per note) by an LLM agent.
- Human quality gating: nothing enters the permanent store unreviewed.
- Positional organisation of notes in a linear order that encodes trains of thought.
- Hybrid retrieval (structural, positional, semantic, lexical) with full provenance.
- Cold retention of originals (`source/.attachments/`), enabling lossless re-processing in the future.
- Multi-user contribution into one shared store with attribution.

Out of scope:

- High-volume corpus search over millions of documents.
- Verbatim document retrieval as the primary access path (the originals layer serves this secondarily).
- Real-time collaborative editing; the store is append-only by design.

The architecture accepts three explicit trade-offs — write-time human effort, bounded throughput, and linear-scan retrieval — in exchange for correctness, durability, and zero operational infrastructure. Section 5 argues why these trade-offs are the right ones at the intended scale.

2 System Overview

2.1 Storage model

The entire knowledge base is a local git repository of plain-text markdown files:

```
slipbox-repo/
├── inbox/            # fleeting captures awaiting distillation
│   └── .attachments/
├── stage/            # review queue: distilled, awaiting approval
│   └── .attachments/
├── store/            # atomic notes — immutable once placed
│   └── .attachments/
├── source/           # bibliography notes: metadata of each source
│   └── .attachments/ # the originals — cold storage beside their bibliography
├── synthesis/        # dated views over the store: answers, overviews, cuts
├── index.md          # nested topic map → entry notes
├── SOUL.md           # process & philosophy, read by agent instances
└── embeddings.db     # sqlite-vec index (derived; outside git)
```

The layout reads as the lifecycle itself — `inbox → stage → store` — with `source/` as the single side collection: a bibliography note holds the provenance metadata, and the provenance itself, in full, sits beside it as that note's attachment (`source/.attachments/<slug>/`). The root contains only note directories.

Two principles govern the model. First, **markdown is the only source of truth**: the vector index and the topic map are declared derivatives, rebuildable from content at any time. Second, **placed notes are immutable**: once a note receives its position, it is never edited or renumbered; corrections are new notes that reference the old ones.

2.2 Ordering: positional identifiers

Every atomic note receives an identifier encoding its position in a sequence of thought, following Luhmann's Folgezettel principle: alternating digit and letter segments, hyphen-separated (`21` , `21-a` , `21-a-1`). A note continuing a thread descends one level; a sibling increments the last segment; insertion between neighbours descends below the predecessor. Sorting is segment-wise (digits numerically, letters alphabetically), yielding a stable linear order in which **identifier adjacency approximates semantic adjacency**. This single invariant is what later makes positional retrieval possible.

Filenames are the bare identifier (`21-a-1.md`); titles live in frontmatter, so renaming a thought never breaks a link.

2.3 The agent

All operations are performed by an LLM agent exposed through skills, a set of read-only inspection tools, and three scheduled jobs. Five skills follow the lifecycle one stage each — `slipbox:capture` , `slipbox:adapt` , `slipbox:review` , `slipbox:link` (alias `slipbox:persist`), `slipbox:search` . Four more are composed from them for the work that spans stages: `slipbox:triage` drives the whole inbox through adaptation in one interactive pass, `slipbox:readapt` mines a source's retained original for **further** atoms, `slipbox:consolidate` maintains the topic map as it grows, and `slipbox:oversight` reviews domain drift from per-

contributor statistics. The division is deliberate — the lifecycle skills are the contract, the workflows are convenience built on top and add no capability of their own. Humans interact conversationally; the agent operates the repository. The capability set ships as a hermes-agent plugin: the conversational agent loads it in-process and operates the store directly through its tools and skills, with no external service in the path. Implementation priority is the retrieval-and-judgement mechanism.

One operation is deliberately **not** the conversational agent's: distillation. Atomisation runs as a dedicated agent with its own model, its own instructions and its own clean context, off the conversation entirely — see Section 8.4. The lifecycle skills that touch it therefore dispatch rather than reason: `slipbox:adapt`, `slipbox:triage` and `slipbox:readapt` hand the work over and report a job, and the conversational agent is told, in the skill itself, that distilling by hand is not its job.

3 The Process

The write path spells **CARP** — Capture, Adapt, Review & Persist; retrieval is the read path outside the acronym. The lifecycle of a piece of knowledge passes through five stages.

3.1 Stage 1 — Capture

A user shares a source through the gateway. `slipbox:capture` extracts its content (web extraction, vision for images), writes a full note into `inbox/` with attribution (`captured_by`), extraction status, and the original reference, and commits. Capture is deliberately dumb: no interpretation, maximum fidelity.

3.2 Stage 2 — Adapt (distillation)

A scheduled **auto-adapt** job processes inbox entries, and it is the **atomiser** — the dedicated distillation agent of Section 8.4, not the conversational one — that does the reading. For each entry the agent:

1. Splits the material into **atomic notes** — one self-contained idea each — placed in `stage/` with review status `pending`; attachments the atoms need travel with them into `stage/.attachments/`.
2. Deduplicates the source semantically against `source/` (by content, not filename) and creates or links a source note carrying the reference.
3. **Moves the original extracted document into** `source/.attachments/`, and records an `original:` pointer in the source note. Nothing is destroyed.
4. Computes embeddings for the new atoms into a dedicated staging vector table, making the content searchable within hours of capture.
5. Runs the placement lookup once, caching the top candidates, the agent's preferred target, and its rationale into the atom's metadata for review — placement is **pregenerated**, not recomputed later. The lookup resolves all the way to a **concrete proposed identifier**: the exact slot the atom will occupy in the store, allocated against the placed identifiers **and** the slots sibling atoms of the same batch have already claimed, so a batch adapted in one pass never collides with itself. The proposal is visible twice over — it is the atom's `id` (in the list form that marks a note as unplaced) and it is the stage filename — so a reviewer reads the intended position without opening anything, and persist is left with a rename.
6. Reads the same lookup a second way, as **atom-level deduplication**: a very close vector neighbour, a reranker score near one, and the judge's answer to "does any candidate express the same idea, not a neighbouring one?" mark the atom as a potential duplicate (`duplicate_of`, with the twin, a similarity, and a verdict). This is a signal, never a decision.

3.2.1 What an atomic note is — the composition contract

Adaptation is governed by an explicit contract, drawn from the slip-box tradition (Luhmann as practised and described by Ahrens), which the judge enforces and the reviewer checks. It begins with a strict separation of note kinds that the repository layout mirrors one-to-one: **fleeting notes** — quick reminders whose only purpose is to empty short-term memory, processed within a day or two and then discarded — are `inbox/`; **literature notes** — brief, selective summaries in one's own words kept next to bibliographic details — are `source/`; **permanent notes** — fully developed, self-contained thoughts — are `store/`. Keeping the three apart is what prevents the store from silting up with half-thoughts.

Not everything captured deserves an atom. Adaptation applies a relevance test rather than archiving isolated facts: does this add to a discussion already under way in the store, or spark a genuine new line of thought? The judge is asked for the **gist** — the underlying principle of an argument, not its supporting detail — and to read past the frame of the source for what the author left out. **Dis-confirming** material earns special weight: contradictions of what the store already holds are among the most valuable atoms, because they open new connections and contrasting threads; the judge is instructed to link them explicitly (`contradicts [[id]]`), and the placement rationale records the tension.

A permanent note must survive the test of time — comprehensible years later, after the original context is forgotten. It therefore contains: the idea in the **writer's own words** (translation forces understanding; a copied quote bypasses the mind and strips the idea of meaning); **full, precise sentences** addressed to an ignorant reader — one's future self — as brief and clear as possible; an **explicit reference** to its source, so every claim is verifiable (the `source` wikilink); and **meaningful connections** — wikilinks to the notes it extends, refines, or contradicts, chosen by asking in which context one would want to stumble upon this note again. It does not contain verbatim quotations, buzzwords, marginalia-style reminders, or more than one argument.

Atomicity is the enforced invariant: **exactly one idea per note**. The physical slip-box guaranteed it with one-sided A6 cards; the digital heuristic is that a note fits one screen without scrolling — a hard length ceiling in the adaptation prompt. Atomicity is what liberates a thought from its original context, so it can be shuffled, compared, and recombined into arguments its source never anticipated. Complexity is never written into a single note; it is **built from connections between simple notes**. When a source carries a complex idea, adaptation deconstructs it into foundational principles, one atom each, and lets the positional identifiers carry the sequence: a follow-on step or elaboration descends below its predecessor (`21` → `21-a`), an independent next thought opens a new top-level number — the same branching-without-renumbering the identifier scheme was designed for. When a cluster grows large, the tradition's **overview note** — a map gathering links to up to about 25 related notes — is the topic map: an `index.md` entry is precisely such a map, and its size threshold is what triggers topic splitting during consolidation. The reviewer's title variants, placement candidates, and re-split action are the human side of the same contract.

3.3 Stage 3 — Human review

Every morning the gateway delivers a status digest: pending atoms grouped **per source**, inbox backlog, extraction failures. The reviewer accepts or rejects **whole per-source batches** with a single decision — per-atom overrides remain available but are the exception. Decisions are stored as frontmatter metadata — the only mutation `stage/` permits.

Beyond accept/reject, review is the system's point of **deliberate shaping**, governed by progressive disclosure: the default flow stays a batch decision with the agent's best guesses

preselected, and alternatives appear only when the reviewer expands an individual atom — never as blocking questions. Three kinds are offered. **Title variants**: two or three alternatives pre-generated during distillation and stored in frontmatter, so review computes nothing — choosing is a metadata edit. **Placement candidates**: the top matches from the shared lookup mechanism (identifier, title, thread), plus a "new thread" option that doubles as naming a new topic-map entry; the choice is pinned as `link_after`, cached candidates are re-validated at persist. **Re-splitting**: a "split differently" action (coarser, finer, or per a textual directive) re-runs distillation against the retained original — an option that exists only because the originals layer does.

Atoms flagged as potential duplicates are lifted out of the default batch acceptance and shown side by side with their twin. The editor chooses: reject (only the new atom is ever removed, never the existing one), accept as a variant (`variant_of [[id]]`, placed next to the original — the right answer for a newer version of the same position), or accept as a distinct thought (a false alarm: a reranker sees similarity of wording, and two near-identical sentences can be contradictory). The threshold is deliberately sensitive, because a false alarm costs one expansion and a missed duplicate costs permanent noise. Only the human editor rejects; the system never drops on its own.

This is the system's quality gate: **the permanent store contains exclusively human-approved content** — with every consequential property of a note (its title, its position, its granularity, its distinctness) decidable by a human at the one moment a human is already reading it.

3.4 Stage 4 — Placement (persist)

A single-instance scheduled job persists accepted atoms — and it does not search: the lookup already ran at distillation. The job consumes the recorded decision and does not allocate either: the identifier was assigned at adapt, so persist **applies the proposed one verbatim** and the move is a rename of `stage/<id>.md` to `store/<id>.md`. Human placement still outranks the machine's — an explicit target, the reviewer's pinned `link_after`, or a `variant_of` verdict discards the proposal and derives a fresh position after the chosen note. Only one case makes persist allocate: the proposed slot having been taken since adapt, which it resolves under its lock by reallocating after the proposal's recorded basis and reporting the substitution. Target validation is trivial because the store is append-only. It then moves the file into `store/` (attachments following into `store/.attachments/`), migrates its embedding from the staging table to the main table (content unchanged, so no recomputation), updates the topic map, and commits with the placement rationale in the message. Rejected atoms are cleaned up together with orphaned attachments and vectors.

3.5 Stage 5 — Retrieval (search)

`slipbox:search` answers questions with a rendered summary in which **every claim cites the specific notes it derives from**, and can quote any cited note verbatim on request. Retrieval is the four-layer mechanism described next.

4 Retrieval Architecture

All lookup — in search, in source deduplication, in placement — runs one shared four-step mechanism:

1. **Structural**: the query is matched against the nested topic map (`index.md`), yielding entry-point identifiers at near-zero cost.

2. **Positional**: from each entry point, the agent bisects the linearly ordered store within the branch (descendants and siblings), exploiting identifier adjacency to sweep up whole threads — including notes sharing no vocabulary with the query, the classic blind spot of pure vector search.
3. **Semantic**: k-nearest-neighbour search over embeddings, scoped to the appropriate vector table, catching associative matches positioned far away.
4. **Judged**: the union of candidates is deduplicated and the agent reads their full content, producing the final ranking. Vectors nominate; a reader decides.

Four refinements govern the mechanism. Topic entries are **bookmarks**: each says "the next ~N notes are about X" until the next entry, so an interval is **derived** at query time as [entry, next-entry) — the right bound is never stored and can never go stale. The bookmark index is hierarchical with one parameter, N: leaves every ~50 notes, a level above every ~300, further levels as the store grows — sixty thousand notes need about two hundred top-level bookmarks and twelve hundred leaves, two levels of ~200 short descriptions each. The rule is identical for a small store and a large one (an interval longer than N splits in two; a level with more than ~N entries gains a level above), and its consequence is the central scaling property: positional search always descends to a **leaf pool of ~50 notes** — the same pool size a small store has — so the cost of bisection is a function of the leaf, a design constant, not of the store. Probe rating in the positional layer is done by a lightweight cross-encoder reranker under a hard probe budget, with a signal-plateau stop that is read by level: a plateau inside a leaf pool means the whole segment is about the same thing and is returned entire — the plateau is a semantic signal, not noise; a plateau at a higher level means descend; only the absence of a leaf hands over to the semantic layer. The semantic layer always searches globally: its candidates are **unioned** with the positional ones, never intersected and never restricted to the structural interval — a multi-layer hit serves only as a soft ranking prior for the judge. And **time is a signal**: every candidate reaches the judge with its age (`created`, the date of the thought; git history as fallback) and, separately, its source's date; on an explicit contradiction the newer position wins and the older is cited as the earlier one — recency as prior, not filter, since an old note is often a foundation rather than a superseded view. An author who knows a note replaces another says so with `supersedes [[id]]`; superseded notes are down-weighted already at the shortlist.

A fourth free channel — exact lexical grep over wikilinks — serves backlink queries. The channels have **decorrelated failure modes**: a query must defeat structure, position, semantics, and lexical match simultaneously to miss. Every result carries provenance (which layer found it), and every summary claim resolves to an immutable, dated, git-audited note.

5 Benefits

Write-time curation eliminates the chunking problem. Mechanical chunking — the unsolved weakness of standard RAG — either splits ideas or dilutes embeddings with unrelated ones. Slipbox segments semantically at ingestion, under review: one atom, one idea, one embedding. Retrieval units are complete by construction.

Immutability makes the index a permanent cache. An embedding is computed exactly once per note and remains valid for the note's lifetime; the index can never silently go stale, and invalidation logic reduces to detecting absence.

Hybrid retrieval from trivial infrastructure. The four decorrelated channels replicate the property that enterprise hybrid-search stacks (sparse + dense + reranker) purchase with substantial machinery — delivered here by a filesystem, one SQLite file, and an ordering convention.

Provenance and verifiability. Answers are not bags of hits: each cited note carries its thread (position), its evidence (source link), its history (git), and its connections (backlinks). The summary is checkable claim by claim.

The originals layer removes the fidelity ceiling. Because originals survive in cold storage outside the retrieval path, extraction quality is no longer capped at what today's model understood: a future `slipbox:readapt` can re-read any retained original with a better model and propose **additional** atoms through the normal review pipeline. The system inherits tomorrow's models for free, without polluting today's search space.

Throughput protected by design. Batch-per-source review compresses the human gate by roughly the atomisation factor, and explicit backpressure (backlog trends, capture-time warnings) makes queue rot visible before it becomes fatal — the failure mode that quietly kills review-gated systems.

Durability and zero lock-in. Plain text under git outlives every proprietary backend; the entire retrieval layer is disposable and rebuildable. The store remains greppable and human-readable with no tooling at all.

6 The Landscape: How Knowledge Is Retrieved Today

Before comparing, it is worth being precise about what the field has converged on by 2026, because Slipbox is not a variant of any of it — it is a different answer to the same question. Five families dominate.

Standard RAG — chunk documents mechanically, embed the chunks, retrieve top-k by vector proximity, generate. It remains the default because it is cheap to build; it also remains the source of the field's best-documented failures. Chunking severs narrative and relational context, so a clause that modifies an earlier section, or a table referencing a previous page, is invisible to the retriever. Two statements can be semantically near yet logically opposite — "safe for normal kidney function" and "dangerous for impaired kidney function" score as neighbours — so contradictions in the corpus are not resolved but smoothed into hedged answers. Indexes drift from their sources: a June 2026 insurance case study describes a claims system citing expired coverage limits because the documents had been updated but the vector index still pointed at old chunks. Most damaging for trust, RAG produces **sourced-looking wrong answers**: a citation beside a paragraph is not proof that every sentence in it is grounded, and surveys find roughly seventy percent of production teams have no systematic evaluation of retrieval quality at all. The remedies the field prescribes — hybrid retrieval, cross-encoder reranking (a legal-tech deployment cut retrieval failures by 41% with a vector-then-reranker pipeline), parent-child chunking, contradiction flagging — are patches over a unit of knowledge that was never designed to be retrieved.

Graph-augmented RAG — Microsoft GraphRAG, RAPTOR, HippoRAG 2, LightRAG — attacks the multi-hop and global-sensemaking weakness by building structure over the corpus automatically: entity–relation graphs with community summaries, recursively summarised trees, passage graphs walked by Personalized PageRank. Systematic evaluations agree on the shape of the result: graph methods win on multi-hop and relational questions, plain RAG still wins on single-hop, detail-oriented ones, and graph construction is expensive, evaluated under heterogeneous protocols, and prone to over-generation — one study found iterative graph retrieval halved a system's willingness to abstain when evidence was insufficient. The structure is real but it is **machine-inferred**, unreviewed, and rebuilt wholesale when the corpus changes.

Agent memory layers — Mem0, Zep/Graphiti, Letta, Cognee — solve a neighbouring problem: letting an agent remember across sessions. Their unit is the automatically extracted fact, stored

in vector, graph, or key-value form; the most technically distinctive of them (Graphiti) adds bi-temporal validity windows so the agent knows when a fact became true and when it was superseded. Their focus is personalisation and conversational continuity, their benchmarks are disputed across vendors, and their extraction runs without a human in the loop — the canonical failure being the assistant that still remembers the client as single six months after his wedding. Notably, practitioner guides now recommend a split: a **markdown vault with humans as first-class authors** for canonical knowledge, and a memory layer only for transient session state.

Agentic filesystem retrieval — the pattern popularised by coding agents: no pre-built index, the agent greps and reads files iteratively, and the "retriever" is whichever shell tool it chose. Two 2026 papers give this teeth: an Amazon study measured agentic keyword search at roughly 94% of RAG-level faithfulness with no vector store, and a PwC study on LongMemEval found grep generally beating vector retrieval inside agent harnesses — with the striking rider that **harness design dominated the algorithm choice**, and how tool output was presented to the model mattered as much as which retrieval ran. Its known weaknesses are token cost on large corpora and blindness to concept-level queries where the right words are unknown.

Corpus-retention assistants — notebook-style tools that keep the originals and re-ask them each session with the newest model — sit apart: maximal fidelity, zero accrual; nothing is learned between questions.

LLM-maintained wikis — the pattern set out by Karpathy in late 2026[1] and elaborated in the `llm-wiki` schema[2] — are the closest neighbour to Slipbox and the most instructive contrast, because they share its premises and diverge on its central mechanism. Three layers: immutable raw sources the LLM reads but never modifies; a wiki, "a directory of LLM-generated markdown files" the LLM owns entirely; and a schema document telling the agent the conventions and workflows. Ingestion is **integration**, not indexing: the LLM "reads the source, discusses key takeaways with you, writes a summary page, updates the index, updates relevant entity and concept pages across the wiki, and appends an entry to the log" — "a single source might touch 10-15 wiki pages". Pages are typed by role (summaries, concepts, entities, syntheses, journal), navigation runs through an index of one-line descriptions, and no embeddings are used at all: at the stated operating point, "~100 sources, ~hundreds of pages", "the index file is enough". The division of labour is explicit — "The human's job is to curate sources, direct the analysis, ask good questions, and think about what it all means. The LLM's job is everything else" — resting on the observation that "the tedious part of maintaining a knowledge base is not the reading or the thinking — it's the bookkeeping". Because pages are rewritten, human corrections need a mechanism to survive regeneration: **pins** record the claim rather than a text diff ("storing the claim rather than a text diff is what makes re-application survive rewording") and are re-checked after each rewrite, kept if still satisfied, surfaced to a human if a newer source contradicts them, flagged if their anchor is gone. The pattern's reported failure modes are as specific as its mechanism: concurrent ingest forks the wiki (one team's batch backfill produced "38% of our pages were semantic near-duplicates"), the index is "a lossy bottleneck" whose one-line summaries "can't surface facts buried deep in page bodies", and drift that no incoming source contradicts goes undetected — the author notes "the dormant case is the piece I have left, and it is specified, not built".

[1] `https://gist.github.com/karpathy/442a6bf555914893e9891c11519de94f`
[2] `https://github.com/tonbistudio/llm-wiki`

7 Comparison: Slipbox Against the Field

The comparison is clearest by axis rather than by product, because Slipbox makes a different choice on each.

Axis	The field	Slipbox
Unit of knowledge	Mechanical chunk (RAG); inferred entity/community (GraphRAG); auto-extracted fact (memory layers); raw file (agentic).	A human-reviewed **atom**: one idea, own words, source-cited, screen-sized. Segmentation happens once, semantically, at write time — the chunking problem is dissolved rather than patched.
Contradictions	Smoothed into hedges by proximity retrieval; resolved by temporal validity windows only in the most advanced memory layers.	Made explicit and permanent: a dis-confirming atom links `contradicts [[id]]`; nothing is overwritten, corrections are new dated notes and `supersedes` links. Every result carries its age; on conflict the judge lets the newer position win and cites the older as earlier. The tension is retrievable as such, not averaged away.
Staleness	Index drifts from source; embeddings computed on content that later changes; wholesale rebuilds.	Notes are immutable, so an embedding is valid for a note's lifetime; markdown is the sole truth and every index a rebuildable derivative; invalidation reduces to detecting absence.
Provenance	Chunk-level or document-level citations; sourced-looking answers whose sentences may be ungrounded.	Claim-level: every sentence of a summary cites immutable, dated, git-audited note identifiers, each carrying its own source reference and thread position. Verifiability is a property of the store, not a prompt instruction.
Structure for multi-hop	Built automatically and expensively (graphs, summary trees); unreviewed; rebuilt on change.	Authored: wikilinks are human-checked connections and positional identifiers encode argument **sequence** — premise → conclusion chains, which is what multi-hop questions actually traverse. Sparser than an inferred graph, but every edge means something.

Axis	The field	Slipbox
Global sensemaking	Community summaries / recursive abstraction generated over the corpus.	The topic map (`index.md`) and overview entries — a maintained hierarchy consolidated periodically, sized by the tradition's ~25-link overview note.
Accrual of knowledge	RAG accrues nothing; memory layers accrue automatically without review; retention assistants re-derive each time.	Accrues deliberately: every atom passed a human gate; originals are retained for later re-adaptation, so accrual improves as models do without polluting the store.
Retrieval mechanics	Single retriever (vector, or grep, or graph walk) plus optional reranker.	Four decorrelated channels — structural, positional, semantic, lexical — funnelled through a reranker into a reading judge; a query must defeat all four to miss.
Infrastructure	Vector databases, graph databases, managed platforms; or a shell and a large token budget.	A git repository, one SQLite file, three local models; disposable retrieval layer; zero external services.

Read this way, Slipbox is best understood as the point the field's own recommendations converge on when they are followed to the end. The "markdown vault with human authors" that memory-layer guides now prescribe for canonical knowledge **is** the store; the cross-encoder reranker that RAG practitioners bolt on afterwards is a first-class stage; the agentic grep-and-read loop that coding tools proved effective is one of four channels; and the harness lesson — that how retrieval results are presented to the model matters as much as the retriever — is why the store returns structured, provenance-tagged atoms rather than bags of text. What no member of the field has is the combination Slipbox is built around: a curated unit, an authored sequence, and originals kept for re-extraction.

The honest converse holds too. Where the field optimises for volume, Slipbox optimises for meaning, and pays in throughput and scale: it will not index a million documents, it will not learn from a conversation without a human seeing the result, and its temporal model is deliberately simple — time as metadata, resolved at read time by the judge, the way a web search engine surfaces dated results — rather than maintained validity windows. It is the right architecture for a person's or a small group's lifetime knowledge — and the wrong one for an enterprise search box.

7.1 The alternative it is closest to: the LLM-maintained wiki

Against the five families above Slipbox differs in kind. Against the LLM-maintained wiki it differs in **choice**, which makes it the comparison that matters. The two agree on almost everything the field disputes: that the unit of knowledge should be written rather than chunked, that raw sources must be retained immutably, that cross-references belong to ingestion rather than query time, that markdown under version control beats a database, and that a human belongs

in the loop. Karpathy's "the wiki is a persistent, compounding artifact — the cross-references are already there" is Slipbox's premise stated in different words.

They then disagree about **mutability**, and every other difference is downstream of it.

Axis	LLM-maintained wiki	Slipbox
Unit	A **page** per source, concept or entity, rewritten as understanding changes; typed by role.	An **atom**: one idea, own words, immutable once placed, carrying a position in a linear order.
What ingestion does	**Integrates.** One source touches 10-15 pages: summary written, entity and concept pages revised, index and log updated.	**Appends.** One source yields a literature note plus atoms; no existing note is edited, ever.
Who decides what enters	The LLM owns the wiki; the human curates sources and directs analysis. Page edits are not individually gated.	The LLM only **proposes**; a human accepts or rejects every atom before it reaches the store.
Corrections	Must survive regeneration: **pins** store the claim (not a diff), are re-applied after each rewrite, and are surfaced when a newer source contradicts them.	Nothing to survive — there is no regeneration. A correction is a new dated note linked `supersedes [[id]]`; the superseded note remains readable and down-weighted.
Retrieval	Index navigation: the agent picks pages from one-line descriptions, then reads them. No embeddings at the stated scale.	Four decorrelated layers — structural, positional, semantic, lexical. The index is one of them, so its failure is insured rather than fatal.
Duplication	A reported failure: concurrent ingest forks the wiki; one batch backfill produced 38% semantic near-duplicates.	A write-time vector check flags a near neighbour as a suspected twin before review; the human gate resolves it. Serialised by file locks.
Cost per source	Low. The LLM does the bookkeeping; the human reads summaries and steers.	High, deliberately. Every atom is read by a person; throughput is bounded by that gate.
Stated operating point	~100 sources, ~hundreds of pages, beyond which index navigation is reported to break down.	Bounded by linear vector scan and the nightly batch, not by the order: bisection stays inside a ~50-note leaf pool at any store size.

The trade is legible in both directions. The wiki is **always current and coherent**: because pages are rewritten, what you read is the system's best present understanding, integrated, with no archaeology required — and it costs far less human attention per source. Slipbox cannot offer that. Its store is a sequence of atoms, not an encyclopedia; the synthesis a wiki page gives

you for free is work the reader or the judge must do at query time, and the price of admission is that a person read every atom.

What Slipbox buys with that price is what rewriting forecloses. An immutable note has a stable identity, so a citation stays valid, an embedding computed once stays valid for the note's lifetime, and git history is the audit trail rather than a record of overwrites. The `pins` mechanism is the clearest illustration: it is an ingenious answer to a problem Slipbox does not have, because a claim only needs re-applying to a page that will be rewritten underneath it. Where the wiki reconciles a contradiction by revising the page, Slipbox keeps both notes and makes the tension itself retrievable — `contradicts [[id]]` is an object in the store, not an edit that removes one side of the disagreement. And where the wiki's index is "a lossy bottleneck" whose one-line summaries cannot surface facts buried in page bodies, Slipbox's structural layer is one channel of four, none of which depends on the others succeeding.

Neither answers the drift the wiki's author leaves open — an assumption that quietly stops being true, contradicted by no incoming source, is invisible to both. Slipbox narrows it slightly by dating every note and letting the newer position win on explicit conflict, but a claim nothing contradicts is not flagged in either system.

The sharpest way to put the difference is one line: **the wiki maintains one synthesis by rewriting it; Slipbox maintains atoms and collects syntheses, dating them.** For the wiki a synthesis is a **state**; here it is a **record**. Slipbox does write syntheses — `synthesis/` holds them (see Section 8.5) — but it writes a new one each time and keeps the old, which is why it can show how its own view of a question changed over a year and the wiki, having overwritten the earlier one, cannot.

The choice between them is therefore not about quality but about what the knowledge is **for**. Where knowledge should read as a current, coherent account of a domain and the cost of a wrong sentence is low, the wiki is the better instrument. Where a claim must still be defensible years later — traceable to a source, to a date, to a reviewer, and to an unmodified original — Slipbox is, and it accepts fragmentation and human effort to get there.

8 Implementation Details

8.1 Repository and formats

Atomic note frontmatter is minimal: `id` (duplicating the filename for self-description and integrity checks), `title`, `source` (wikilink, list-valued when needed), `created`. Deliberately absent: tags (position and the topic map serve that role), modification dates (immutability plus git), explicit outgoing links (they live in the body as wikilinks). Source notes carry `title`, `author`, `type`, `reference` (ISBN/DOI/URL), `accessed`, and `original` (pointer to the cold copy); their filename is an author–title slug — sources are footnotes, not thoughts, and receive no position. In practice the source record is the one place the minimalism is relaxed, because a footnote is only useful if it is complete: the implementation also carries `date`, `topic`, `tags`, and the `attachments` that travel with the note into its cold-storage directory.

The **shape** of `id` is itself the note's placement state, which makes every note self-describing and every mis-filing detectable by inspection alone. A scalar Folgezettel means placed and immutable (`store/`); the same identifier wrapped in a one-element list means **proposed but not yet earned** — the slot an atom in `stage/` will occupy if review accepts it; a UUID means positionless by nature (`source/`). Nothing outside the identifier needs to be consulted to tell the three apart.

8.2 Originals — `source/.attachments/`

`source/.attachments/<source-slug>/` holds the full extracted text and attachments of every processed capture. The layer is **never embedded and never enters the lookup mechanism**; it is reachable only deliberately (direct show, explicit grep). Text is committed to git; heavy media above a size threshold is git-ignored or pruned after distillation. It serves three roles: it removes the extraction-fidelity ceiling (future re-distillation with better models), it powers review-time re-splitting, and it doubles as an injection quarantine: raw captured content — the untrusted artifact — is opened only intentionally, never as ambient retrieval context.

8.3 Semantic layer

The system runs on a small, fixed set of model **classes**. A large generative model is the **judge**, reserved for the irreplaceable operations: atomisation, final candidate judgement, and the cited summary (reference: a quantized ~24B generalist, strong in the system language). It is not necessarily the model the user is talking to, nor even one model: atomisation runs on the dedicated agent of Section 8.4, configured independently, and only the interactive cited summary is delegated to the conversational model that is already loaded. Where the judge reference is a large generalist chosen for the summaries a human reads, the atomiser is chosen for throughput on an unattended batch — the two are the same role only when a deployment says so. A lightweight cross-encoder **reranker** (reference: bge-reranker-v2-m3) scores query–document pairs jointly — attention runs across both texts, yielding precision unavailable to vector comparison, at a per-pair cost that places it mid-funnel: probe rating, topic matching, shortlisting to a dozen. A multilingual **embedder** (reference: bge-m3, 1024-dimension dense vectors) encodes each note exactly once. The cost funnel: the embedder and the structural/positional layers nominate, the reranker narrows, the judge reads in full and decides. Resource strategy: the GPU belongs entirely to the conversational model, and CARP runs on CPU — it is asynchronous and low-priority, so a nightly batch tolerates single-digit tokens per second (the judge runs as a GGUF quantization; a mixture-of-experts judge shortens the batch several-fold). Interactive search keeps its cheap layers in-process on CPU and delegates its generative steps to the host agent's already-loaded model — the GPU is never contended and locking stays file-scoped. The quality budget still belongs to the judge: weak embeddings degrade gracefully, insured by the other channels, while a weak judge degrades every stage and the final product. The vector store is sqlite-vec: one file, zero servers, three separate `vec0` tables — main store, source, staging — so that "similar thought", "matching source", and "fresh material" remain distinct query spaces, never post-filtered from one polluted ranking. The database records per row the note path and a content hash, and an `embedding_meta(model, dim)` table; startup and every scheduled run compare file state against the database — a fresh clone triggers a full rebuild, individual gaps trigger point re-embedding, and a model mismatch is a hard error forcing a full reindex. Search scoping per operation: placement queries the main table only (using the staged vector as the ready-made query); source deduplication queries the sources table; search queries all spaces separately and merges only in the summary, marking staged hits as not yet situated.

8.4 The atomiser — distillation as a dedicated agent

Atomisation is the operation the whole store's quality rests on, and it is the one operation that must **not** run on the conversational agent. Letting the host distil inline has three defects, and they are structural rather than incidental: it blocks the conversation for the length of the work; it distils inside whatever context the conversation had already accumulated, so the same source yields different atoms depending on what was said before it — the composition contract stops being a constant; and it makes unattended operation impossible, because a cron entry

has no conversation to reason in. The nightly job was, for exactly this reason, the one piece of automation that could not actually be automated.

So distillation is a **dedicated agent**: its own model, its own instructions, its own empty context, running off the conversation. Every trigger routes through it — the tool, the slash command, the lifecycle skills, and the cron sweep — so there is one distillation path and it behaves identically however it was started. Manual triggers hand the work off and return a job identifier at once; the atoms appear as the agent commits them. What makes an in-process job registry sufficient, rather than a durable queue, is that every step already commits: the repository **is** the record, so a registry lost to a restart costs visibility, never work.

The deployment configures two things, and only two: which model distils, and what it is told to do. Both come from the host's plugin configuration, because they are deployment decisions rather than repository ones — and the second is not a prompt tweak but a redefinition of what the store means, since the instructions **are** the composition contract. The default backend keeps the model in-process, so no captured content leaves the machine; the alternative delegates to the host's own model lane, whose provider routing and credentials the plugin never sees, and whose model can only be steered when the operator has granted that trust explicitly.

The agent's authority is deliberately narrow: it **proposes**. It returns a structured plan — a literature note and a list of atoms with their placements, scopes and title variants — which is validated and bounded before any of it reaches the repository, and then executed by the same deterministic operations every other path uses. Nothing about the store is entrusted to the model: a hallucinated placement target costs that one atom and the rest of the batch proceeds, a plan that will not parse is re-asked with its own error quoted back, and the ceiling on atoms per source makes the relevance test mechanical. Re-adaptation is the same agent pointed at cold storage instead of the inbox, shown the atoms already distilled from that source and told not to repeat them — which is also where the originals layer's quarantine finally pays off in full: the untrusted artifact is read by the one component whose entire output is validated before it is trusted.

8.5 Syntheses — a materialised view over the store

`synthesis/` holds dated documents that **cite** atoms: the answer to a question, an overview note binding a cluster in the tradition's sense, a comparison of two threads, a topical essay. A synthesis carries no position — a UUID, like a source note — because it belongs to no train of thought; it cuts across them. It has its own vector table and a small frontmatter: `title`, `question`, `created`, `cites`, `supersedes`. The analogy is a materialised view in a database: derived from the tables, refreshable, and never mistaken for one.

One rule carries the whole design. **A synthesis is never the proof of a claim — the proof is always the atoms it cites.** It is a navigational object. This is precisely where the LLM-maintained wiki differs, and where its page **is** the proof. The consequence is that a synthesis may be written automatically and without human review while every guarantee of §3.3 survives intact: an atom is evidence and therefore cannot enter unseen, whereas a synthesis asserts nothing on its own authority, and anything a reader or the judge takes from it must still resolve to an atom. Review is spent where it buys something.

Retrieval — a fifth channel, consulted before the four rather than beside them. The other layers answer "which notes are about this"; this one answers "has this road been walked before". A query is matched against the synthesis's `question`, and a hit means the hops are already made, the threads gathered and the citations placed. The judge then does not compose from nothing — it checks the **delta**: how many atoms have been placed in the cited threads since the synthesis was written. That test is a date comparison and a prefix match, costing

nothing. An empty delta means the earlier answer still stands; a non-empty one means the judge reads only those atoms and writes a new synthesis superseding the old. The result is a wiki's speed on repeated questions with none of its cost, because nothing is overwritten to get it.

Structure — the third geometry. Positional identifiers express threads and the topic map expresses topics, but until now nothing expressed a cross-thread cut except a single wikilink in an atom's body. A synthesis citing twenty atoms from five threads **is** that cut, and it now has somewhere to live instead of being pressed into `index.md` as a "see also" or into the store as a hub-note that is not one idea. Three structural layers, three geometries: threads, topics, cuts. It is also what keeps `index.md` honest — the map surplus goes here, and the topic map stays pure bookmarks.

Reading budget. A judge under a probe ceiling reads one synthesis to orient itself rather than fifteen atoms, and opens atoms only when it needs the evidence.

Analysis. Because re-synthesis writes a new dated note rather than editing an old one, two answers to the same question a year apart can be diffed, and the store can report how its own view changed — a capability no member of the field surveyed in §6 has, for the simple reason that they all overwrite. Three cheap measures follow from the same data, all grep-and-arithmetic with no model in the loop. **Drift**, per synthesis, is the count of atoms in its cited threads it does not account for; high drift marks a re-synthesis candidate, and it is the natural fourth scheduled job beside adapt, persist and the digest — weekly, because drift is a slow signal and re-synthesis is a judgement. **Coverage** is its inverse: an atom no synthesis cites is knowledge never integrated into an answer, one cited by twenty is load-bearing. That is a better drift signal for a domain charter than per-contributor capture statistics, because it maps what was **asked** onto what was kept rather than counting what was dumped in. And a synthesis citing two atoms joined by `contradicts` is expected to name that tension in prose: the edge is already visible to grep, and the synthesis makes it visible to a reader.

The direction of flow is one-way and load-bearing: atoms compose into syntheses, never the reverse. A synthesis cannot become an atom, so no second-order loop exists in which the system feeds on its own summaries; at most it is the reason an atomiser, reading a conversation transcript later, notices something worth distilling.

8.6 Concurrency and automation

Three scheduled jobs — auto-adapt, single-instance persist, and the morning digest — plus manual skills all take `flock`-based locks on the resources they touch (`inbox/`, staging, the vector database, the repository for commits), waiting or skipping when contended. The atomiser takes a lock of its own, under a name no ordinary operation uses: a whole distillation is serialised against another distillation, while the steps **inside** it still acquire the ordinary locks themselves. Reusing those names at the outer level would deadlock the pipeline against its own inner steps, since advisory locks conflict between two open descriptions of a file even within a single process. Backpressure is first-class: the status tool reports backlog counts and week-over-week trend per contributor, and capture acknowledges with a warning once the pending queue crosses a threshold.

8.7 Read-only tools

A flat inspection set mirrors the storage areas — show, inbox, stage, source, store listing in positional order, thread view (ancestors/siblings/descendants), backlinks, topic map, status, per-note git log, and scheduler status — implemented as pure reads: no commits, read-only database connections, one shared identifier comparator.

That set doubles as a **deployment mode**. A single switch (`SLIPBOX_READONLY`) reduces an instance to its read surface — the inspection tools, the search-and-quote path, and the `search` skill — while every write tool, every write skill, the write half of the command line, and the setup and commit hooks are withheld; freshness still reports, so a read-only instance can still say that its view is stale. The enforcement point is deliberately **registration**, not the call: a tool the agent was never handed cannot be misused, cannot be prompted into firing, and needs no per-operation guard to audit — the whole write group simply never enters the schema. This is what lets a store be queried by an agent that must not modify it, while another instance or the scheduled jobs remain the only writers.

8.8 Multiple stores

One instance can serve several knowledge bases at once. A registry (`SLIPBOX_REPOS`) names them and fixes an order whose first entry is the default, so every operation that names no store still has one; each tool carries an optional store argument and echoes back the store it acted on, which keeps a mistargeted write visible in the result rather than only in the history. Leaving the registry unset collapses the whole mechanism back to a single store, unchanged.

Isolation is free rather than engineered, and that is the point: every piece of per-store state — the vector database, the locks, the topic map, the domain charter — already lives **under the store root**, so two roots share nothing without a separating mechanism having to exist. The same property made the change small: the operations layer already threaded a root through every call, so what was added was a registry, a resolver, and a loop. Everything that runs unattended — setup, the three scheduled jobs, the health and scheduler reports, and both session hooks — iterates the registry rather than assuming one store, while the conversational slash commands act on the default.

This is the substrate for visibility boundaries, and only the substrate: it makes private and shared material genuinely separate stores in one agent's reach. What it does not yet supply is the policy on top — promotion of a note from a private store into a shared one, and the review that would gate it.

8.9 Multi-user operation

One shared store serves all gateway users, and the arrangement is coherent for a reason deeper than locking: **the author of every atom is the agent**. The composition contract — one idea, own words — is executed by the judge at adaptation, and the same judge proposes each note's position, so the "continuator of thought" that Folgezettel presumes is one mind: the agent's. Users are the **editorial board**: they supply raw material (capture) and verdicts (review). This dissolves the classic multi-user objections at the root. Peer review carries no social friction — rejecting an atom corrects a machine draft, not a colleague's prose, and `captured_by` records provenance of interest, never authorship — so any editor may review any batch. The taxonomy is one mind's map, not a committee compromise. Mechanically, immutability and single-instance placement keep sharing safe (nobody overwrites, identifiers are assigned in one place, source deduplication works across contributors), and a single system language — with originals' key quotes preserved in source notes — keeps the store coherent under multilingual input.

Agent authorship makes mind-continuity a requirement across **time** rather than across people: swapping the judge model is changing authors mid-book. The store's identity is therefore defined by the composition contract and the judge prompts, not by model weights — the prompts are versioned in the repository alongside `SOUL.md`; review smooths residual differences of voice, and re-adaptation lets a newer author add rather than rewrite.

The one genuinely multi-user problem that remains is **domain drift**: a base founded for one domain being fed content from outside it. It is handled by the same pattern as duplicates — the system signals, a human decides. An administrator writes a **domain charter** into `SOUL.md` (what the base is, what it is not, what counts as adjacent), which finally operationalises the relevance filter of the composition contract: it names **which** discussion content must add to. Adaptation, which reads the material anyway, classifies each source against the charter — `scope: in / adjacent / out`, one sentence of rationale, into the staging metadata at no extra model cost. In review, `out` drops from batch acceptance and requires an explicit decision; `adjacent` passes flagged, deliberately, because adjacency is where dis-confirming material lives and must not be cut by an automaton. Roles complete the picture: **contributor** (capture), **editor** (review — anyone, any batch), **admin** (charter, rules, and pattern-level decisions such as muting a source or spinning off a store for a topic that outgrew the domain, informed by per-contributor scope statistics from the status tool). Federation of stores remains a tool for one thing only: visibility boundaries between private and shared material — not for coherence, which agent authorship already provides.

9 Risks and Limits

Honest constraints remain. Placement is heuristic and permanent — mitigated by review-time pinning and by wikilinks carrying connections independent of position, but a misplaced thread relies on the semantic channel. Prompt injection through captured content is a real, softly-mitigated risk: note content is data by policy, and the originals layer quarantines raw material, but no hard boundary exists. Scale is bounded not by the positional order — the bookmark index keeps every bisection inside a leaf pool of ~50 regardless of store size — but by linear vector scans, the nightly adaptation batch (linear in captures), and git over hundreds of thousands of files. Each has a documented escape hatch, and they share one: because every index is a derivative of the markdown, a large store can move its index into a relational database — `notes` with a materialised sort key so an interval is an indexed `BETWEEN`, `bookmarks(level, start, end, description)` instead of a nested list, typed `links` (`extends`, `contradicts`, `supersedes`, `variant_of`) queryable in both directions, and the vector tables in the same file — without touching a single note; the index may then grow arbitrarily, because nothing reads it whole. Multi-user operation resolves more cleanly than it first appears — the sequence **is** single-authored, by the agent; review needs no owner because editors correct machine drafts; the taxonomy is one mind’s — leaving two genuine residuals: visibility is only half-solved — several isolated stores can now be held by one instance, and a store can be exposed read-only to an agent that must not write it, but the promotion of a note from a private store into a shared one, and the review that ought to gate it, do not exist — and prompt injection through capture scales linearly with contributors, which keeps the charter, the review gate, and the data-not-instructions rule load-bearing. Finally, the human gate bounds throughput by design; the system suits deliberate practice, not firehoses.

10 Conclusion

Slipbox occupies a deliberately contrarian position: it spends intelligence where today’s tools save it (ingestion) and saves it where they spend it (query time). The result is a knowledge base whose every entry is atomic, reviewed, positioned, attributed, and permanent; whose retrieval is layered and verifiable; whose originals survive for better models to reread; and whose entire stack — files, git, one SQLite file, a local embedding model — will still be operable decades from now. For its intended scale, that combination is not available from any current tool — and, as the landscape review shows, it is the point toward which the field’s own best practices are converging.